

DAY 1

GCP Foundations

Storage & Ingestion

BigQuery Basics

01

Today's session flow

1	Storage services on GCP — what each one is for	30 min
2	Buckets, lifecycle rules, storage classes & ACLs	35 min
3	Hands-on:	25 min
4	Pub/Sub architecture: topics, subscriptions, push vs pull	30 min
5	Schema Registry & contract-driven ingestion - Hands-on:	25 min
6	Apache Beam model & Dataflow Flex Templates	30 min
7	BigQuery architecture: Dremel, Jupiter, Colossus	25 min
8	Datasets, tables, views & materialized views	30 min
9	Partitioning, clustering & hands-on lab	35 min
10	Review & Quiz	20 min

Storage options on GCP

Cloud Storage

Object storage for files, backups, and staging data

Spanner

Globally distributed, strongly consistent relational database

Cloud SQL

Regional managed MySQL, PostgreSQL, or SQL Server

Firestore

Document-model NoSQL for app and mobile workloads

Bigtable

Wide-column store for high-throughput, low-latency workloads



Cloud Storage



Spanner



Cloud SQL



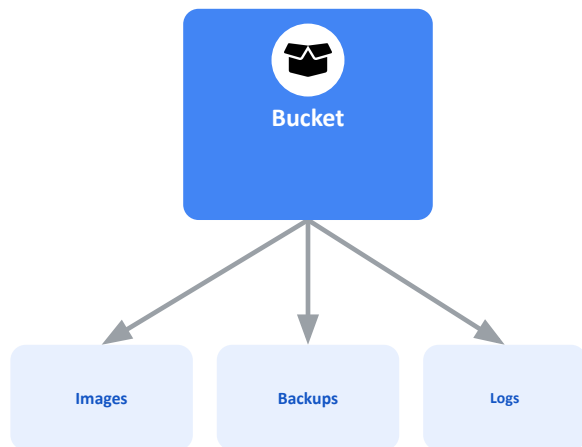
Firestore



Bigtable

Cloud Storage

Object storage for files, backups, and data lake staging — any size, any format



Data model

Objects (blobs) in flat buckets — no schema, no rows or columns



Best for

Media files, backups, data lake staging, static site hosting



Scale

Exabyte-scale per bucket, virtually unlimited objects

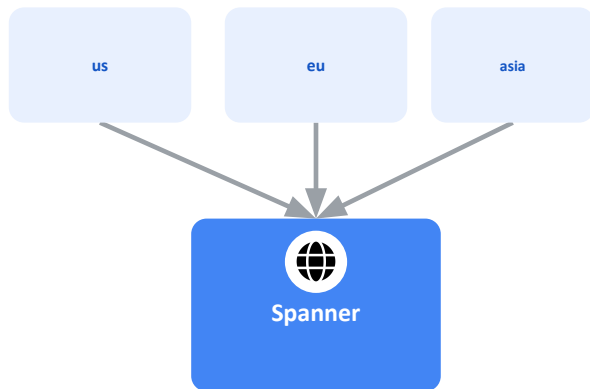


Key features

Storage classes, lifecycle rules, versioning, strong consistency

Spanner

Globally distributed, strongly consistent relational database



Data model

Relational (SQL) — tables, rows, joins, ACID transactions



Best for

Multi-region apps needing SQL plus horizontal scale



Scale

Scales across regions — petabytes, 99.999% SLA



Key features

TrueTime consistency, automatic sharding, zero-downtime resizing

Bigtable

Wide-column NoSQL for high-throughput, low-latency workloads



Data model

Wide-column — sparse rows keyed by a single row key



Best for

Time-series, IoT telemetry, ad-tech, high-throughput feeds



Scale

Petabyte-scale, millions of operations per second

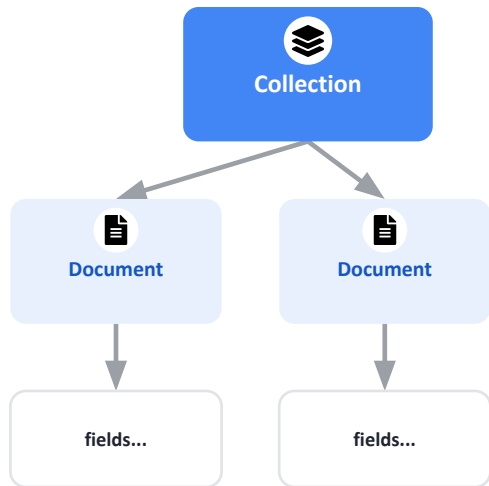


Key features

Single-digit ms latency, HBase-API compatible, linear autoscaling

Firestore

Document-model NoSQL for mobile, web, and realtime apps



Data model

Documents grouped into collections — flexible, JSON-like



Best for

Mobile/web apps, real-time sync, offline-first experiences



Scale

Scales automatically to millions of concurrent clients

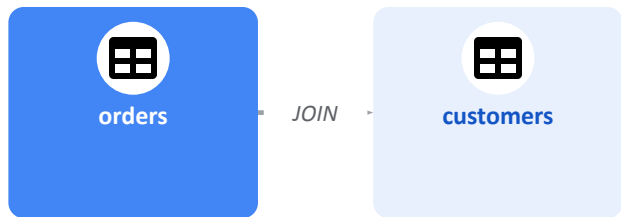


Key features

Realtime listeners, offline support, native mobile/web SDKs

Cloud SQL

Fully managed MySQL, PostgreSQL, and SQL Server



Data model

Relational (SQL) — traditional tables, rows, joins



Best for

Lift-and-shift apps, regional OLTP, familiar SQL workloads



Scale

Vertical scaling plus read replicas — regional, not global



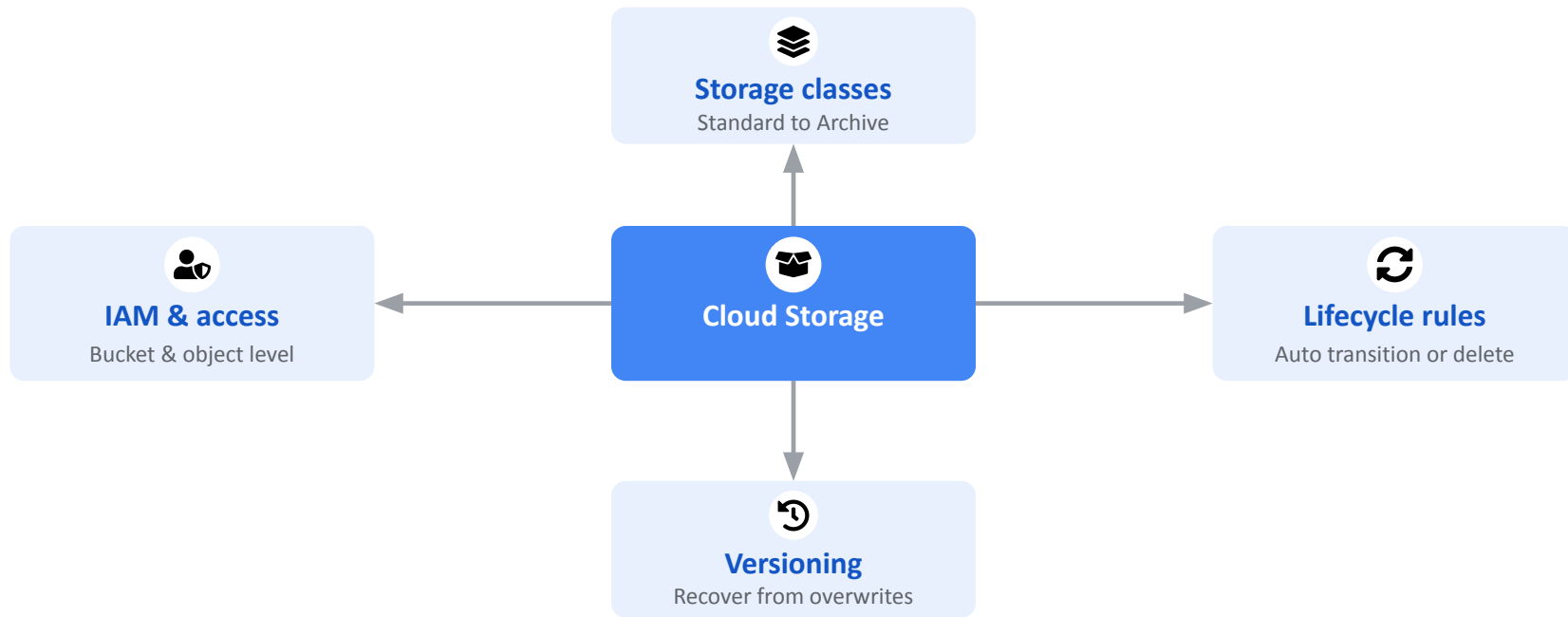
Key features

Automated backups, patching, high-availability failover

Which storage service fits?

Service	Data model	Consistency	Best for
Cloud Storage	Objects / blobs	Strong (per object)	Files, media, backups, data lake staging
Spanner	Relational (SQL)	Strong, global	Multi-region OLTP needing SQL + horizontal scale
Cloud SQL	Relational (SQL)	Strong, regional	Traditional apps, lift-and-shift databases
Firestore	Document (NoSQL)	Strong, per document	Mobile/web apps, real-time sync
Bigtable	Wide-column (NoSQL)	Eventual / tunable	Time-series, IoT, high-throughput feeds

What Cloud Storage does, at a glance



One service, four pillars — object storage that's cheap, safe, governed, and self-managing.

Cloud Storage: key features at a glance



Storage classes

Standard, Nearline, Coldline, Archive — priced by access frequency



Lifecycle rules

Automate class transitions or deletion as objects age



Object versioning

Keep prior versions to recover from accidental overwrites



IAM & signed URLs

Fine-grained access control, plus time-limited shareable links



Strong consistency

Read-after-write consistency on every object, globally



Native integrations

Direct access from BigQuery (BigLake), Dataflow, and Vertex AI

Cloud Storage classes at a glance

Class	Minimum storage duration	Retrieval cost	Typical use
Standard	None	None	Frequently accessed, hot data
Nearline	30 days	Low	Backups, data accessed ~monthly
Coldline	90 days	Moderate	Disaster recovery, data accessed ~yearly
Archive	365 days	Highest	Long-term retention, compliance archives

Lifecycle rules move objects between classes automatically as they age.

One object's journey through its lifecycle



A single lifecycle rule can encode this entire journey — no manual intervention required.

Lab: Cloud Storage bucket + lifecycle rule

1

Create the bucket

```
gsutil mb -l asia-south1 gs://<your-bucket-name>
```

2

Enable object versioning

```
gsutil versioning set on gs://<your-bucket-name>
```

3

Apply the lifecycle rule

```
gsutil lifecycle set lifecycle.json  
gs://<your-bucket-name>
```

4

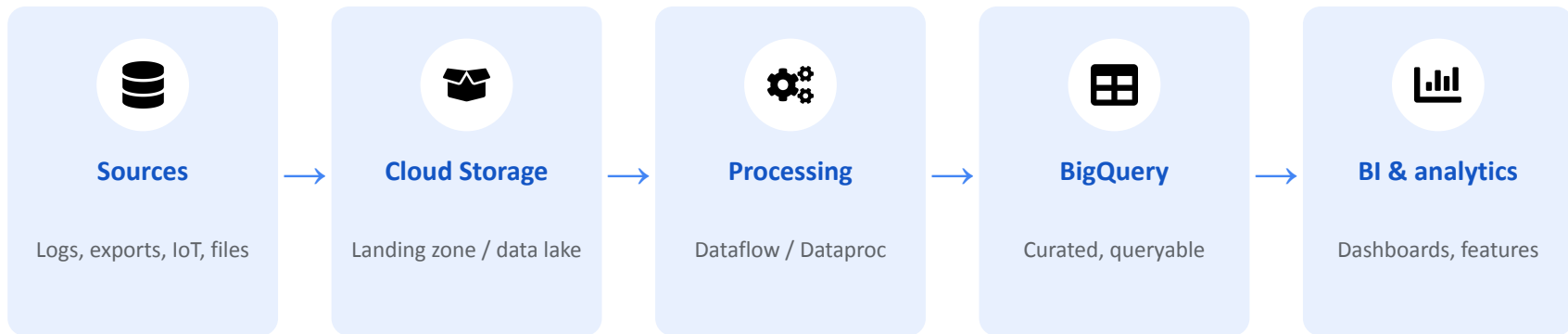
Verify the rule is active

```
gsutil lifecycle get gs://<your-bucket-name>
```



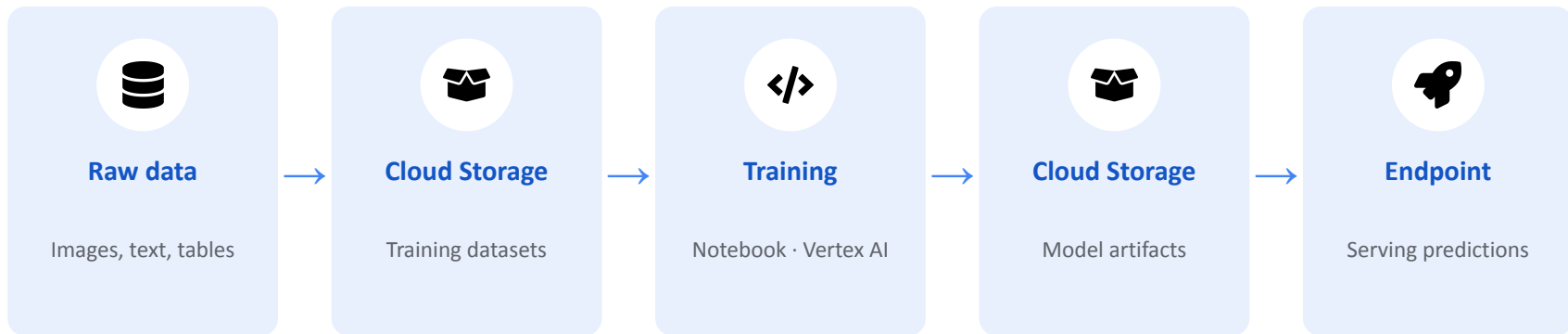
Expected outcome: objects older than 30 days auto-transition to Coldline storage.

Where it fits: a data engineering pipeline



Cloud Storage is almost always the first stop for raw data — cheap, durable, and format-agnostic before anything gets transformed.

Where it fits: a data science / ML workflow



Notice Cloud Storage shows up twice — once for the training data going in, once for the model artifacts coming out.

Pub/Sub: topics & subscriptions

Topic

Named resource that receives and stores published messages

Subscription

A stream of messages from one topic, delivered to a subscriber

Pull delivery

Subscriber explicitly requests messages — good for batch-style consumers

Push delivery

Pub/Sub sends messages to an HTTPS endpoint — good for event-driven services



Publisher



Topic

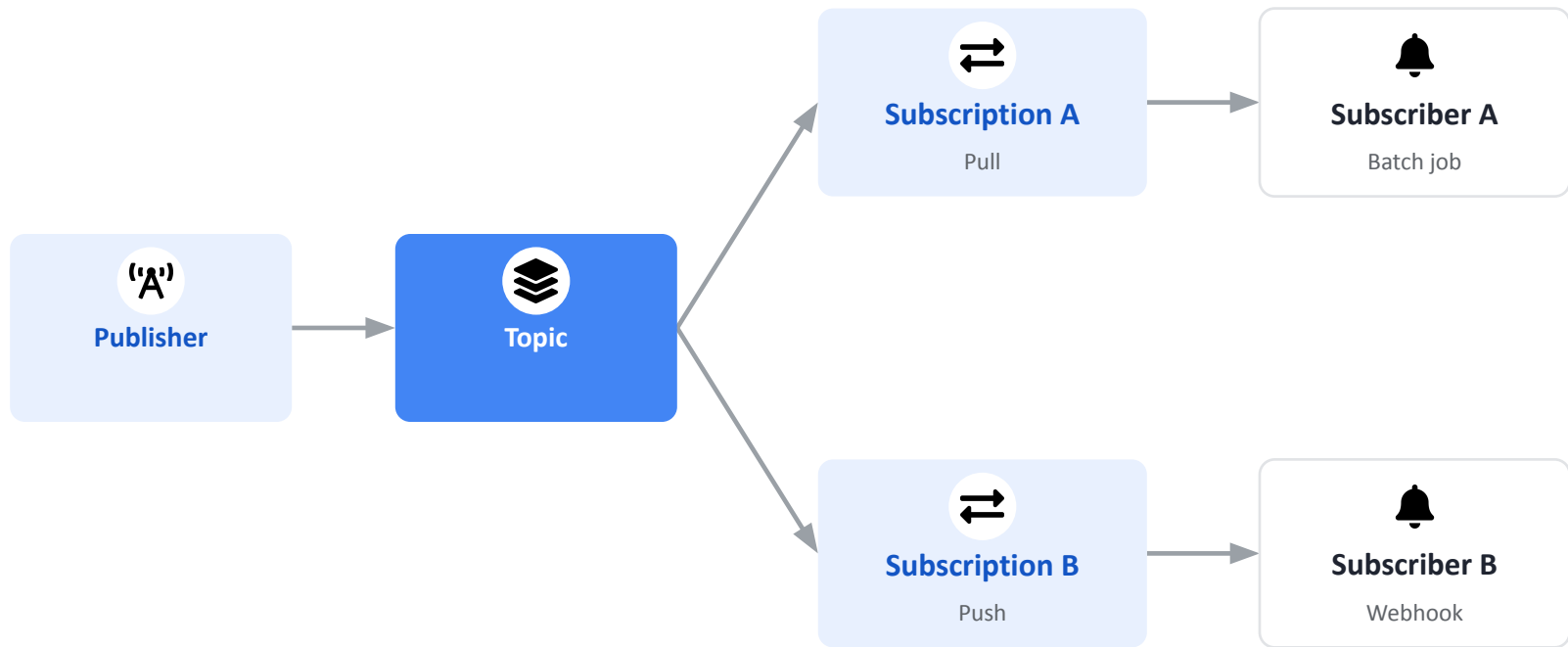


Subscription



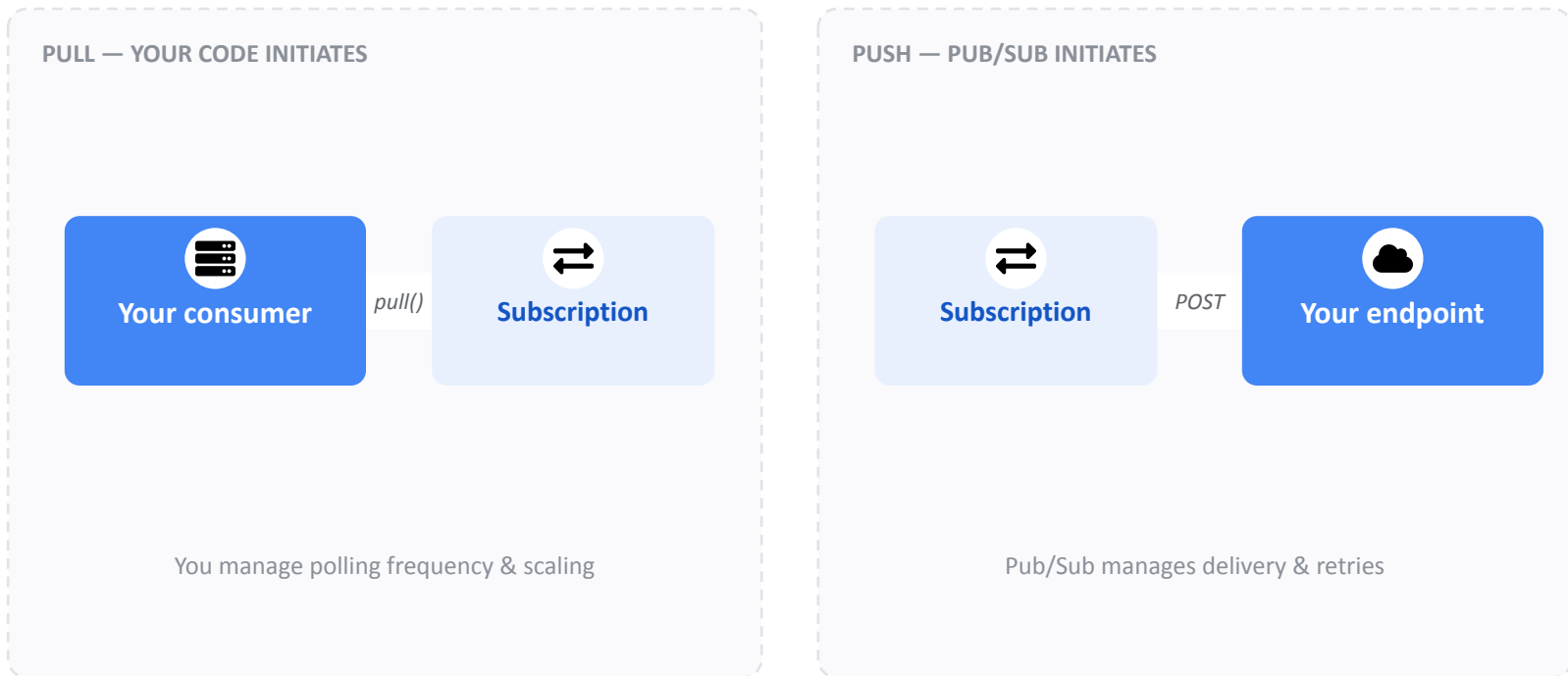
Subscriber

Pub/Sub fan-out: one topic, many subscriptions



Each subscription gets its own independent copy of every message published to the topic.

Pull vs push: who dials the phone?



The only real difference is who dials the phone — pull has your code call Pub/Sub; push has Pub/Sub call your code.

Commands: topics, subscriptions & publishing

1

Create a topic

```
gcloud pubsub topics create orders-topic
```

2

Create a pull subscription

```
gcloud pubsub subscriptions create orders-pull-sub  
--topic=orders-topic
```

3

Create a push subscription

```
gcloud pubsub subscriptions create orders-push-sub  
--topic=orders-topic  
--push-endpoint=https://myapp.a.run.app/receive
```

4

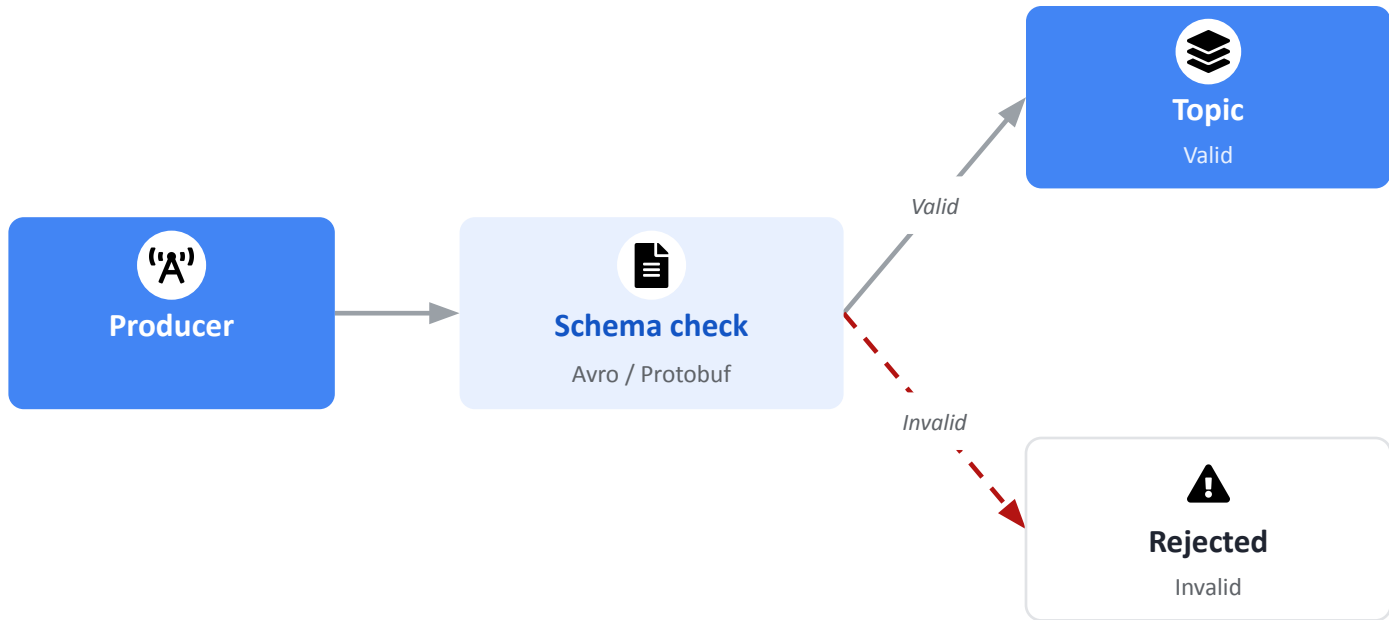
Publish a message

```
gcloud pubsub topics publish orders-topic  
--message='{"order_id": 123}'
```



Expected outcome: the same message arrives at both subscriptions — one delivered via pull, one via push.

Schema Registry: enforcing a contract at ingestion



Malformed messages never reach the topic — the contract is enforced at publish time, not discovered three hops downstream.

Commands: schemas & contract-driven ingestion

1

Define an Avro schema

```
gcloud pubsub schemas create order-schema --type=AVRO  
--definition-file=order_schema.avsc
```

2

Create a topic that enforces it

```
gcloud pubsub topics create orders-topic  
--schema=order-schema --message-encoding=JSON
```

3

Publish a valid message

```
gcloud pubsub topics publish orders-topic  
--message='{"order_id": 123, "amount": 49.99}'
```

4

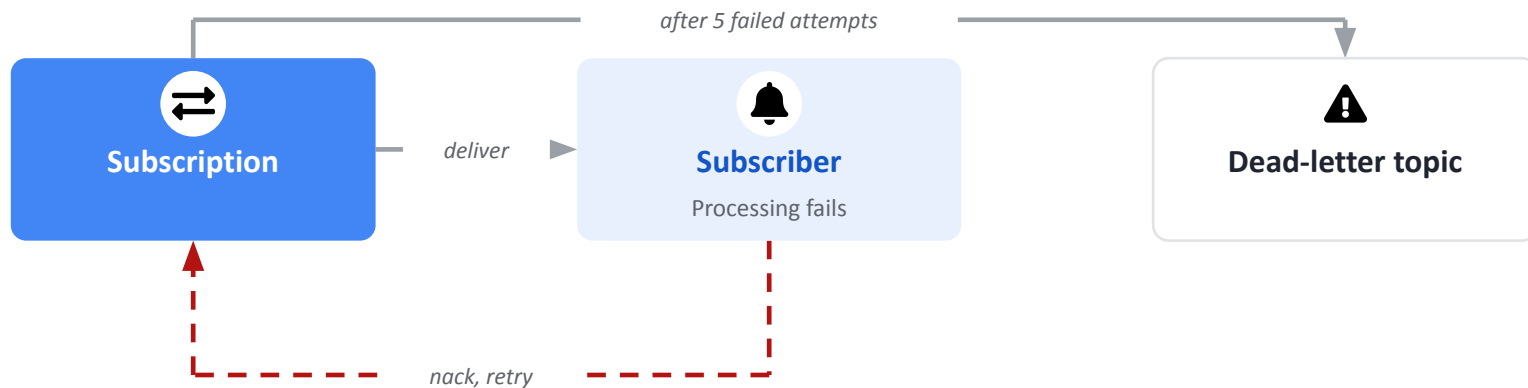
Watch an invalid message get rejected

```
gcloud pubsub topics publish orders-topic  
--message='{"order_id": "not-a-number"}'
```



Expected outcome: the valid message publishes successfully; the malformed one is rejected before it ever reaches the topic.

Dead letter queues: what happens when delivery fails



After the maximum delivery attempts, Pub/Sub stops retrying and routes the message to a dead-letter topic instead of losing it silently.

Commands: configuring a dead-letter policy

1

Create the dead-letter topic

```
gcloud pubsub topics create orders-dlq
```

2

Attach a dead-letter policy to the subscription

```
gcloud pubsub subscriptions update orders-pull-sub  
--dead-letter-topic=orders-dlq --max-delivery-attempts=5
```

3

Grant Pub/Sub permission to publish to the DLQ

```
gcloud pubsub topics add-iam-policy-binding orders-dlq  
--member=serviceAccount:service-PROJECT_NUMBER@gcp-sa-pub  
sub.iam.gserviceaccount.com --role=roles/pubsub.publisher
```

4

Inspect the dead-lettered messages

```
gcloud pubsub subscriptions pull dlq-inspect-sub  
--auto-ack --limit=10
```



Expected outcome: messages that fail delivery 5 times land in orders-dlq instead of retrying forever or being silently dropped.

Apache Beam: one model, many runners

PCollection

An immutable, distributed collection of data flowing through the pipeline

PTransform

An operation applied to a PCollection — map, filter, group, aggregate

Runners

The same Beam pipeline can execute on Dataflow, Spark, or Flink, unchanged

Why it matters

Write the pipeline once in Beam's SDK — Dataflow is just one place to run it



PCollection



PTransform

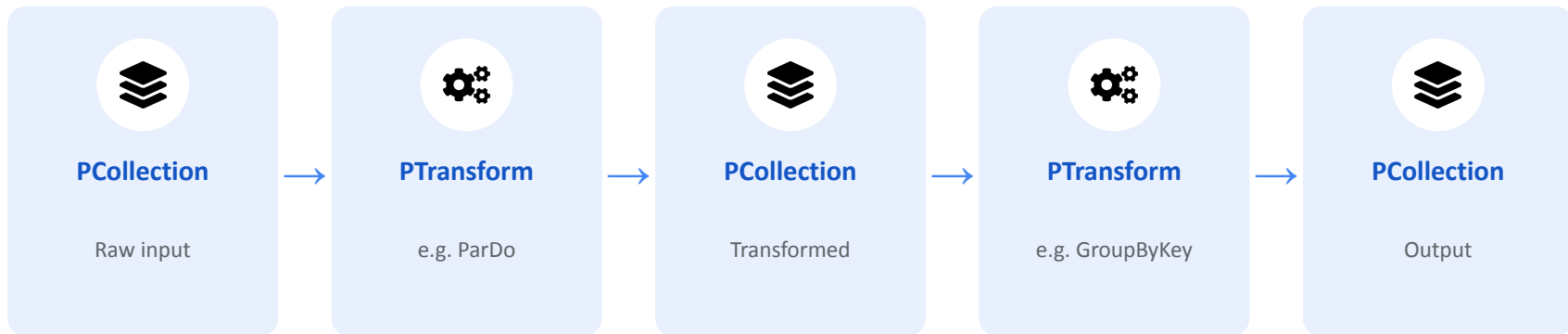


Any runner



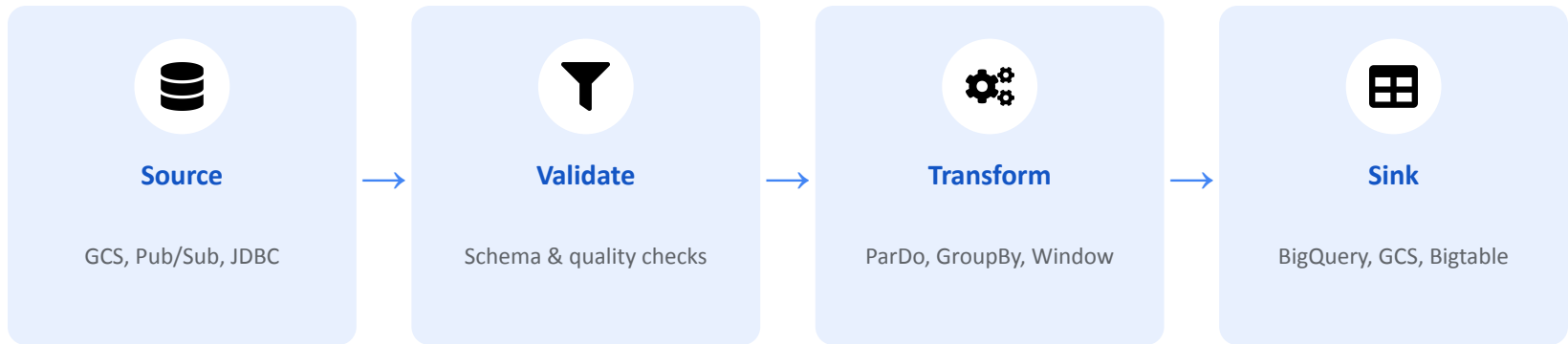
Write once

The Beam programming model, in one picture



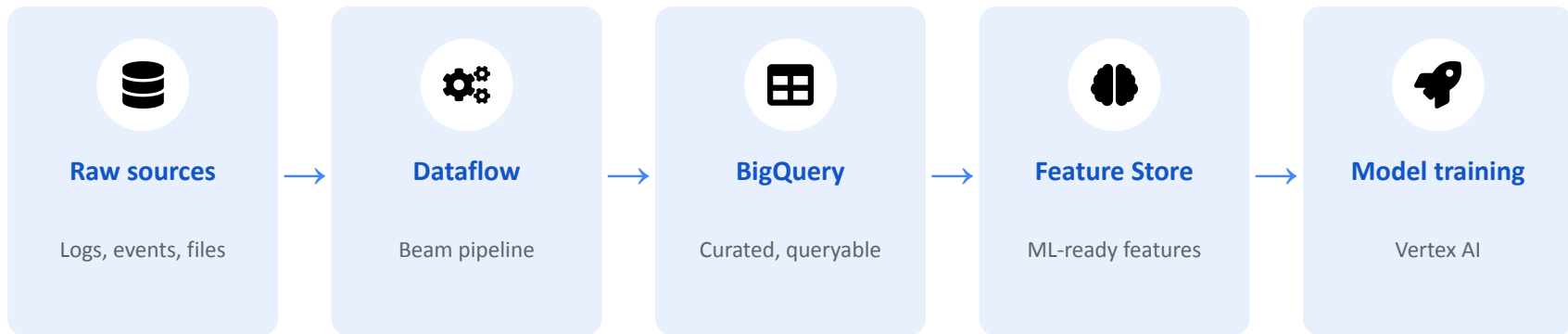
Every transform produces a brand new PCollection — nothing is mutated in place, which is what makes a pipeline safely parallelizable.

A pipeline's shape: source, transform, sink



One pipeline definition, two runtimes — batch for bounded data, streaming for unbounded, windowed data.

Where Dataflow fits: from raw data to data science



Dataflow is the bridge — it's usually the thing standing between raw, messy data and anything a data scientist can actually use.

Templates: packaging a pipeline for reuse

The problem

A data engineer writes a pipeline — but an analyst needs to run it with different parameters

A template

A pre-built, parameterized pipeline anyone can launch without touching code

Launched how

Console, gcloud, REST API, or a Composer/Scheduler job — no SDK needed to run one

Why it matters

Separates who builds a pipeline from who runs it



Packaged



Parameterized



One-click launch



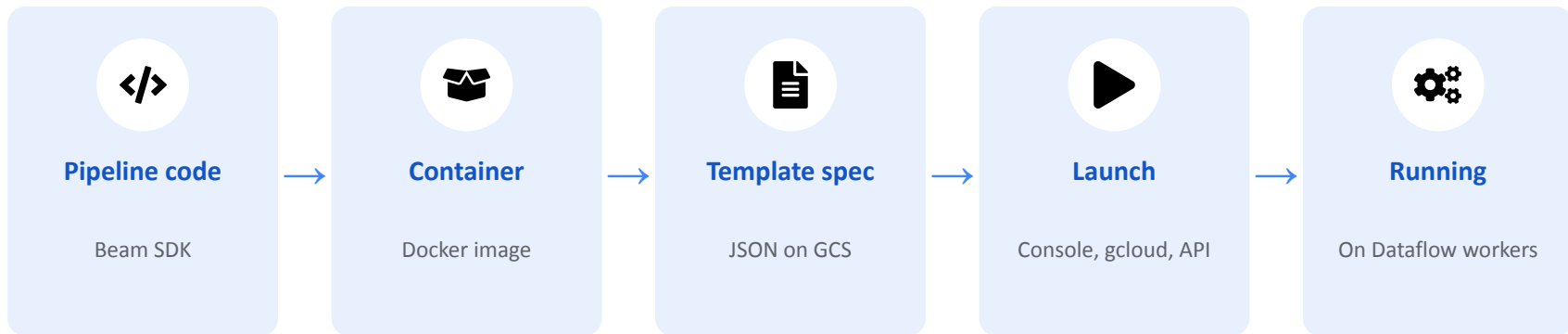
Build vs run

Classic templates vs Flex Templates

	Classic templates	Flex Templates
How it's packaged	A staged execution graph on GCS	A full Docker container image
Flexibility	Limited — fixed structure set at staging time	Full custom logic, any dependency, any SDK feature
Best for	Simple, standard pipelines	Complex pipelines needing custom containers or libraries
Current guidance	Legacy — being phased out	Recommended for all new pipelines

If you're building a new pipeline today, build it as a Flex Template — classic templates are the legacy path.

How a Flex Template actually runs



Anyone can launch this pipeline — they never need to see or touch the code, just the parameters.

BigQuery's architecture

Dremel

The distributed query execution engine — fans a query out across thousands of workers

Colossus

Google's distributed file system — stores your table data, decoupled from compute

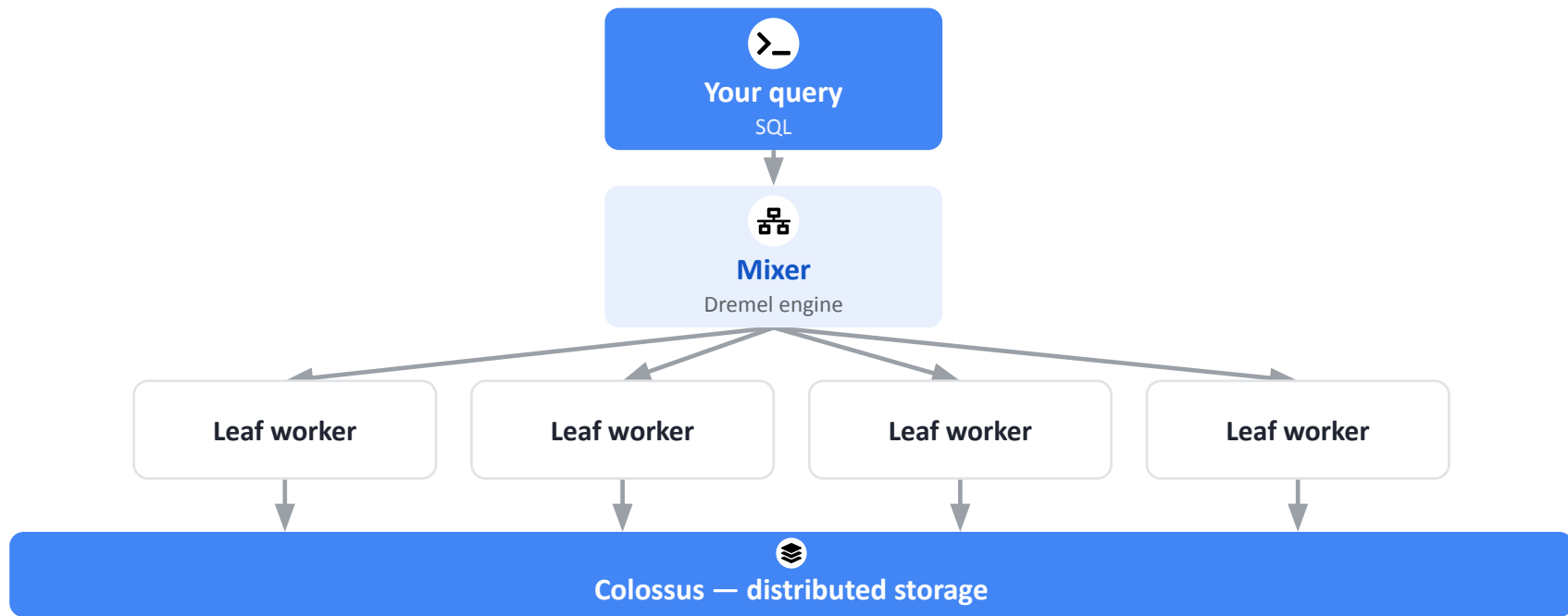
Jupiter

The datacenter network fabric moving data between Colossus and Dremel at high bandwidth

Why it matters

Storage and compute scale independently — you don't provision either one

How a BigQuery query really runs



Dremel fans a query out across many workers in parallel — all reading the same data from Colossus over the Jupiter network.

Why decoupled storage & compute matters



Independent scaling

Storage grows and shrinks with your data; compute scales per query, separately



Pay for what you scan

On-demand pricing bills bytes scanned, not a provisioned cluster sitting idle



No indexes to manage

There's no cluster to size and no index to build, tune, or rebuild



Elastic by default

The same architecture serves a 1GB table and a 1PB table or more if you need

Datasets, tables & views

Dataset

Top-level container for tables, views, and access controls — like a schema

Table

Structured data with a defined schema; the unit of storage and billing

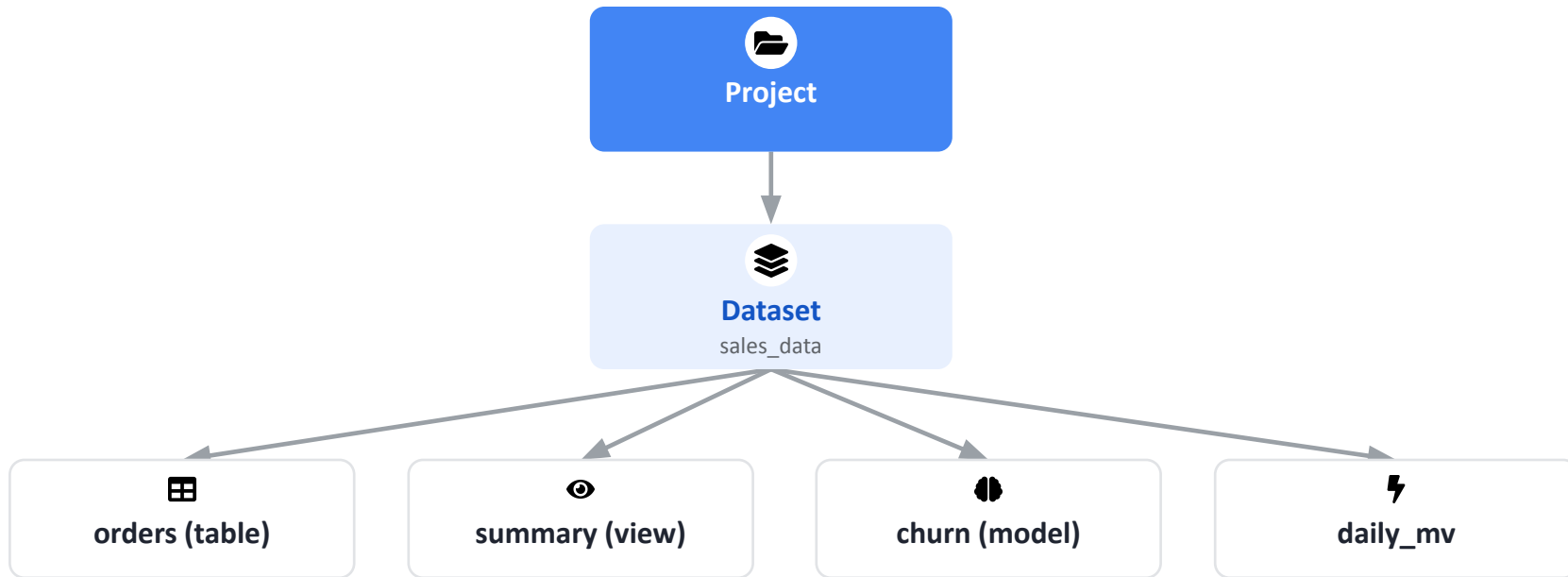
View

A saved SQL query, re-executed against live data every time it's queried

Materialized view

Precomputed, cached results that BigQuery refreshes automatically

Where things live: project, dataset, table



A dataset is the container — tables, views, materialized views, and even models all live inside one, side by side.

Datasets: the container for your tables

A dataset is the top-level container inside a project — it holds tables, views, and access controls.

Location is set once, at creation, and can't be changed afterward — choose it deliberately.

PROJECT



sales_data

location: asia-south1

```
CREATE SCHEMA `myproject.sales_data`  
OPTIONS (location='asia-south1')
```

Tables: structured data with a schema

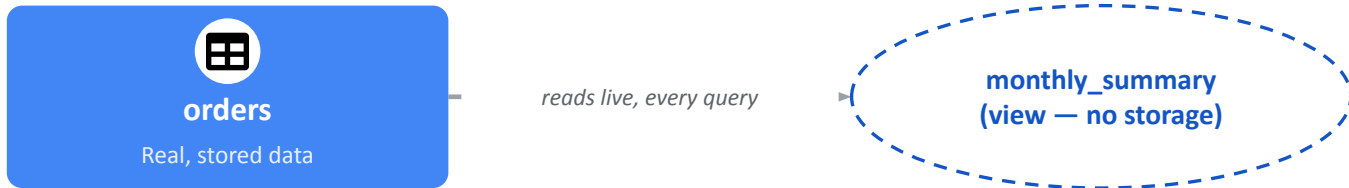
A table stores real data with a defined schema — it's the unit of storage and billing.

order_id	customer_id	amount	order_ts
1042	C-7781	49.99	2026-07-11
1043	C-9034	18.50	2026-07-11

```
CREATE TABLE sales_data.orders (  
  order_id INT64, customer_id STRING, amount NUMERIC, order_ts TIMESTAMP)
```

Views: a saved query, not stored data

A view is just saved SQL — it has no storage of its own and re-runs live, every time.



```
CREATE VIEW sales_data.monthly_summary AS
SELECT DATE_TRUNC(order_ts, MONTH) AS month, SUM(amount) AS total
FROM sales_data.orders GROUP BY month
```

Materialized views: a saved query that's actually cached

A materialized view stores its results and refreshes automatically — queries hit the cache, not the source.



orders

Real, stored data

auto-refreshed incrementally



monthly_summary_mv

Cached results, auto-refreshed

```
CREATE MATERIALIZED VIEW sales_data.monthly_summary_mv AS
SELECT DATE_TRUNC(order_ts, MONTH) AS month, SUM(amount) AS total
FROM sales_data.orders GROUP BY month
```


Table vs view vs materialized view

	Table	View	Materialized view
Storage	Stores data	No storage — just a query	Stores precomputed results
Freshness	Always current	Always current, re-run live	Auto-refreshed, near real-time
Query cost	Pay for data scanned	Pay for the underlying query, every run	Cheaper — scans cached results
Best for	Primary data storage	Reusable logic, simpler SQL	Frequently-run aggregations

Materialized views can also be queried directly by BI tools for fast dashboards.

Partitioning & clustering

Partitioning

Splits a table by date, timestamp, or integer range — prunes by partition

Clustering

Sorts data within partitions by up to 4 columns — prunes further within each one

Why combine them

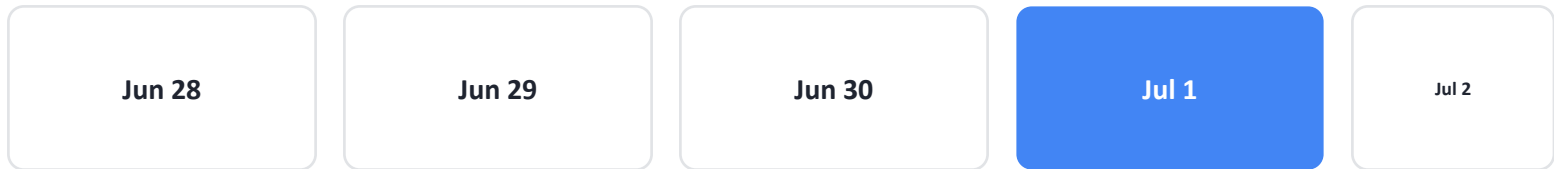
Partitioning prunes coarse ranges; clustering prunes fine-grained blocks inside

Cost impact

Both reduce bytes scanned — the main driver of on-demand query cost

Partitioning: splitting a table by date or range

Partitioning splits a table into physical chunks — a query can skip entire chunks it doesn't need.



WHERE DATE(order_ts) = '2026-07-01' → only the Jul 1 partition is scanned

```
CREATE TABLE sales_data.orders_p
PARTITION BY DATE(order_ts)
AS SELECT * FROM sales_data.orders
```

Clustering: sorting data within each partition

Clustering sorts rows inside a partition by up to 4 columns — pruning further, within the block you already narrowed down.

PARTITION: JUL 1

customer A–F

customer G–M

customer N–S

customer T–Z

```
CREATE TABLE sales_data.orders_pc
PARTITION BY DATE(order_ts) CLUSTER BY customer_id
AS SELECT * FROM sales_data.orders
```

Pruning in action: what a filtered query actually scans

Table: sales_data.orders_pc — PARTITION BY date, CLUSTER BY customer_id

	Jun 28	Jun 29	Jun 30	Jul 1	Jul 2
A–F					
G–M					
N–S				Scanned	
T–Z					

WHERE date='Jul 1' AND customer_id BETWEEN 'N' AND 'S' → 1 of 20 blocks scanned instead of the whole table.

Lab: partitioned + clustered table

1

Create a partitioned table

```
CREATE TABLE sales.orders_p PARTITION BY DATE(order_ts)
AS SELECT * FROM sales.orders
```

2

Add a clustering column

```
CREATE TABLE sales.orders_pc PARTITION BY DATE(order_ts)
CLUSTER BY customer_id AS SELECT * FROM sales.orders
```

3

Run a pruned query

```
SELECT * FROM sales.orders_pc WHERE
DATE(order_ts)='2026-06-01' AND customer_id='C1042'
```

4

Compare bytes scanned

```
bq query --dry_run --use_legacy_sql=false '<same query on
orders vs orders_pc>'
```



Expected outcome: the partitioned + clustered query scans a fraction of the full-table scan's bytes.

Designing a well-organized table



Pick location once, carefully

Dataset location can't be changed later — decide deliberately, not by default



Partition by date if you can

Most tables have a natural date column — start pruning there first



Cluster by your filter column

Whatever you filter or join on most often is your best clustering key



Materialize what's expensive and frequent

A view queried often with a costly aggregation belongs as a materialized view instead

Choosing the right ingestion pattern



Batch

Scheduled loads of bounded data — daily exports, file drops



Streaming

Continuous, unbounded events via Pub/Sub and Dataflow



CDC

Capture row-level changes from a source database as they happen



Bulk file load

Direct load into BigQuery or Cloud Storage for one-off transfers

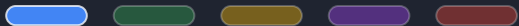


DAY 1 RECAP QUIZ

Storage, ingestion & BigQuery basics

10 questions · 10 minutes · Live on Kahoot

[Join at kahoot.it](https://kahoot.it)



Day 1 complete

Tomorrow: Day 2 — BigQuery Advanced & Governance

Questions?